Lớp: **IT003.R17**

BÁO CÁO KẾT QUẢ THỬ NGHIỆM

Thời gian thực hiện: 01/10 – 10/10/2026

Sinh viên thực hiện: Bùi Thị Giang – MSSV: 25520461

Môi trường đo: CPU (Intel(R) Core(TM) 5 210H); RAM (16.0 GB); Window 11(version 25H2); 64-bit operating system, x64-based processor; Visual studio; cờ /02

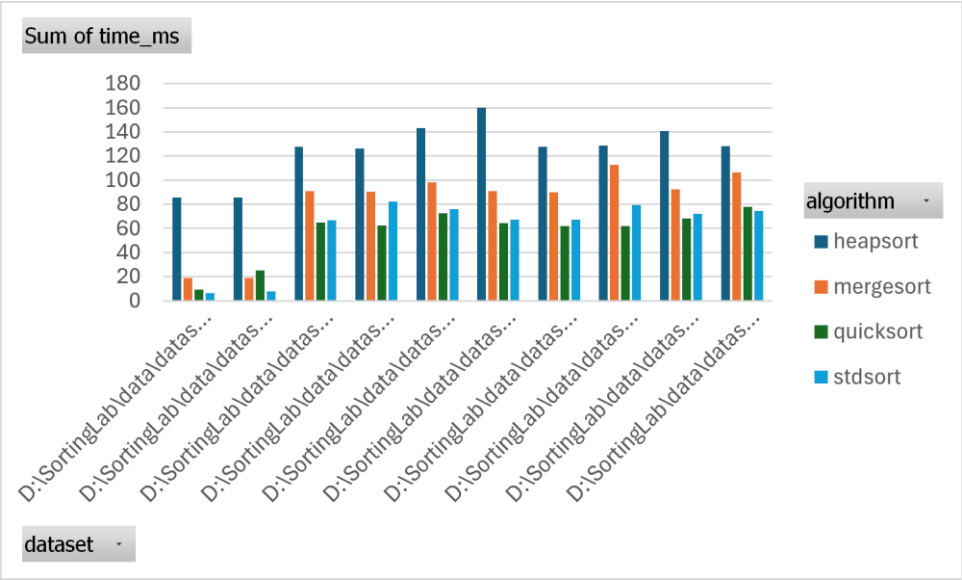
Nội dung báo cáo:

I. Kết quả thử nghiệm

1. Bảng thời gian thực hiện *(đơn vị: mili-giây — ms; n = 1.000.000 phần tử mỗi bộ dữ liệu; dòng 1 = dữ liệu tăng dần, dòng 2 = dữ liệu giảm dần, dòng 3–10 = ngẫu nhiên)*

Dữ liệu	Quicksort	Heapsort	Mergesort	stdsort
1	9.043	85.663	18.845	6.522
2	24.934	85.516	18.647	7.732
3	64.841	127.881	90.692	66.576
4	62.413	126.139	90.532	82.385
5	72.367	143.149	98.231	75.906
6	64.121	159.926	90.776	67.167
7	61.736	127.886	89.953	67.427
8	61.714	128.442	112.861	79.358
9	68.15	140.556	92.27	71.978
10	77.664	127.962	106.506	74.238
Trung bình	56.6983	**125.3156**	80.9313	59.9289

2. Biểu đồ (cột) thời gian thực hiện



II. Kết luận

Trên bộ dữ liệu 1 triệu số thực, sort() của C++ và Quicksort luôn có thời gian thực hiện nhanh nhất (trung bình lần lượt khoảng 59.93 ms và 56.70 ms). Mức chênh lệch không quá lớn ở hầu hết các trường hợp.

Mergesort có độ phức tạp O(n log n) ở mọi trường hợp. Khi dữ liệu đã có thứ tự, bước gộp luôn lấy phần tử từ cùng một nửa nên bộ dự đoán rẽ nhánh của CPU và bộ nhớ đệm hoạt động hiệu quả; còn với dữ liệu ngẫu nhiên, các nhánh so sánh khó đoán hơn. Trên dữ liệu ngẫu nhiên, Mergesort chậm hơn Quicksort và sort() vì tốn thêm bộ nhớ phụ và phải sao chép dữ liệu qua lại giữa mảng gốc và mảng tạm ở mỗi lần gộp.

Heapsort chậm nhất trong cả 4 thuật toán (trung bình khoảng 125.32 ms) dù cùng độ phức tạp lý thuyết O(n log n) như Mergesort — nguyên nhân khả dĩ nhất đến từ việc heap truy cập bộ nhớ không liên tục (mỗi thao tác sift-down nhảy giữa các chỉ số 2i+1, 2i+2 cách xa nhau trong bộ nhớ), gây nhiều cache miss hơn so với Mergesort/Quicksort vốn duyệt tuần tự mảng .

Nhận xét chung: độ phức tạp Big-O lý thuyết giống nhau (O(n log n)) không đồng nghĩa tốc độ thực tế giống nhau — hằng số ẩn phía sau Big-O (số lần truy cập bộ nhớ, tính cục bộ của dữ liệu trong cache, số phép so sánh/hoán đổi thực tế) tạo ra chênh lệch rõ rệt.

III. Thông tin chi tiết — link GitHub

Repository: https://github.com/25520461-collab/sorting-benchmark

Trong repo GitHub công khai nêu trên có:

1. Báo cáo — file PDF này và bản Word gốc, trong thư mục **report**

2. Mã nguồn — chương trình sinh dữ liệu, cài đặt 4 thuật toán và chương trình benchmark, trong thư mục **src**

3. Dữ liệu thử nghiệm — 10 bộ dữ liệu đã dùng và file kết quả đo được trong thư mục **data** và **results**